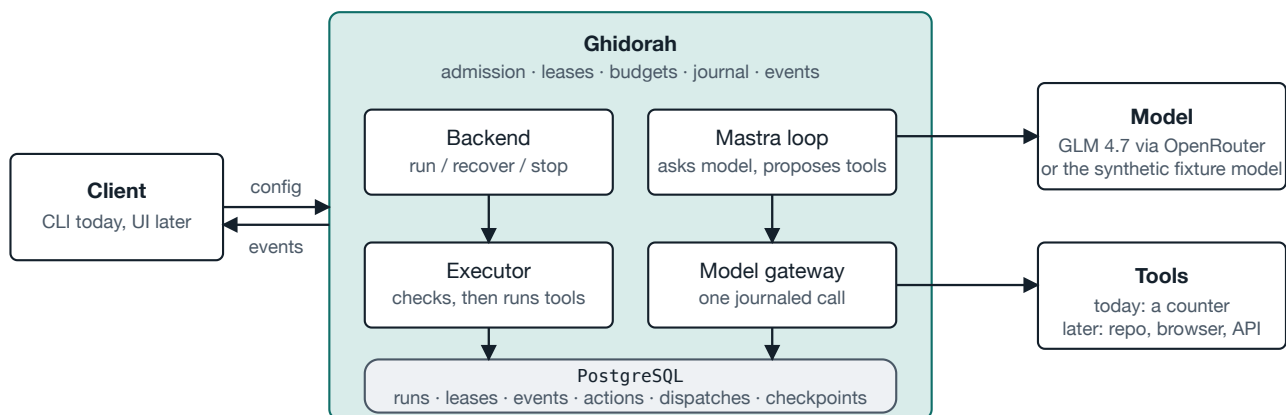


How Ghidorah works

A plain explanation, with pictures, of what the harness does today: how a run starts, how the model is called, how a tool is executed, and why a crash cannot corrupt the record.

The big picture

Ghidorah is the backend behind Mettle. It does not think. Mastra runs the agent loop and asks a model what to do next. Ghidorah decides whether that step is allowed, writes it down before it happens, executes it, writes the result, and only then tells the client. PostgreSQL is the single source of truth.



Everything the client sees was first committed to PostgreSQL. Nothing is inferred from model text.

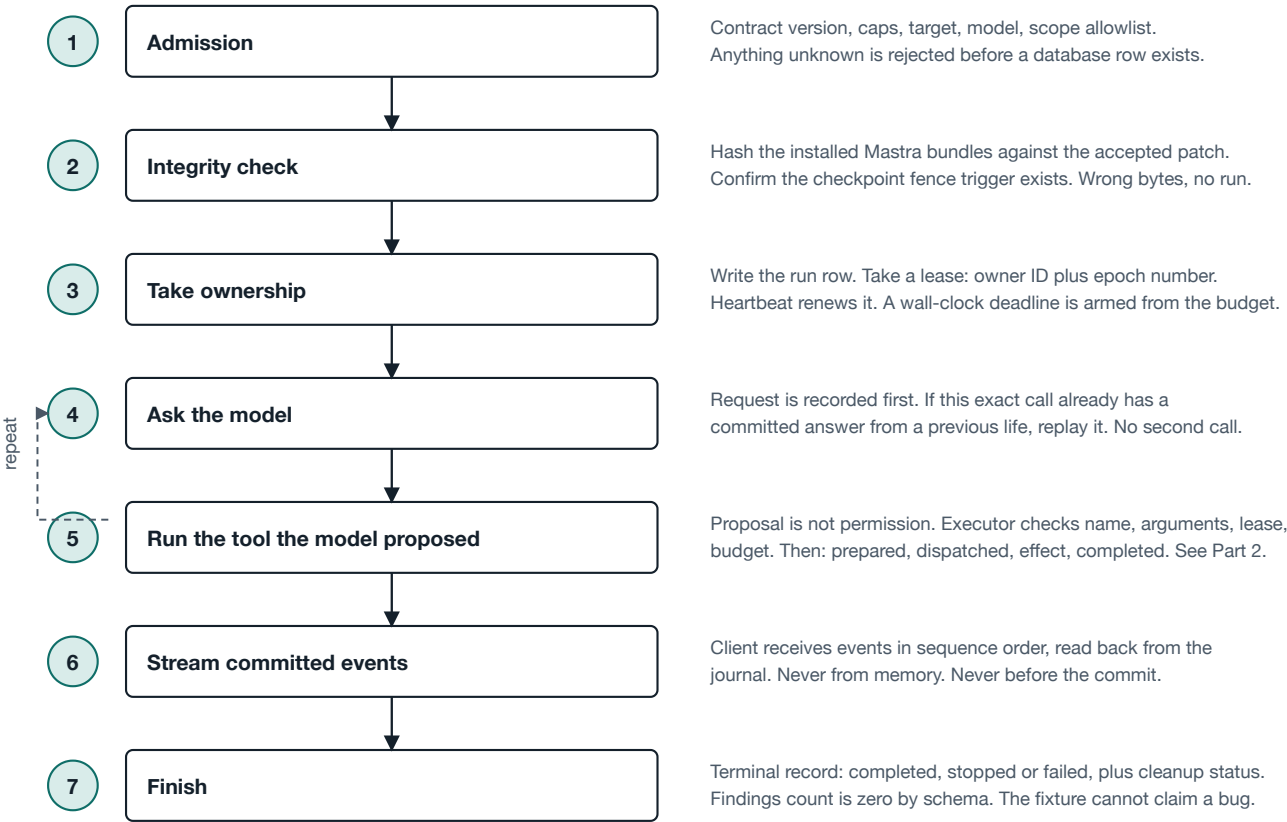
Figure 1. Mastra proposes. Ghidorah checks, records, executes, records again, then reports.

The one rule

If something may have happened but its result was never committed, Ghidorah stops and asks a human. It never guesses and never silently retries.

One run, start to finish

Every run follows the same seven steps. Today the only allowed job is the synthetic counter: increment once, read once, finish. That job is deliberately trivial so that the surrounding machinery can be tested with real process kills.

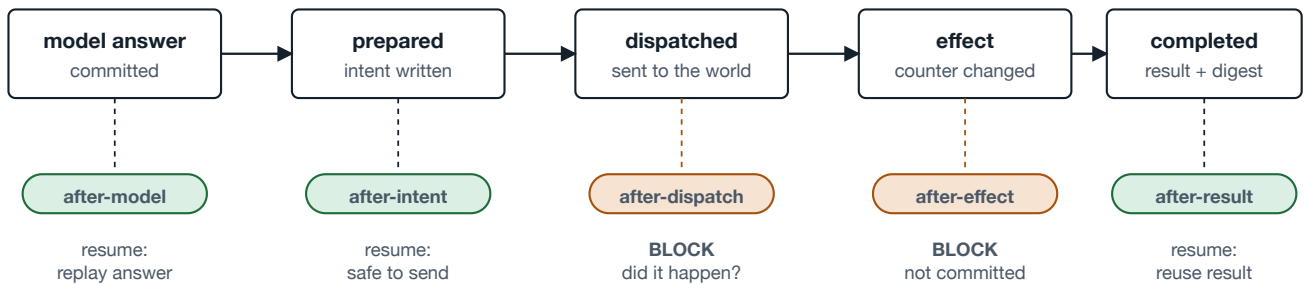


Result of the fixture: counter = 1, five steps, six synthetic tokens, zero findings.

Figure 2. Steps 4 and 5 repeat until the model says it is done or a limit intervenes.

How a tool call is recorded

A tool call is the one place where the agent touches the world. Ghidorah writes four records around it, each in its own database transaction. Between those records are the five places a process can die. Tests kill a real worker at each one.



Green: recovery continues from the journal. Orange: the outcome is unknown, so recovery refuses to proceed.

Figure 3. The five fault points. Two of them are uncertain and block on purpose.

Why block instead of retry

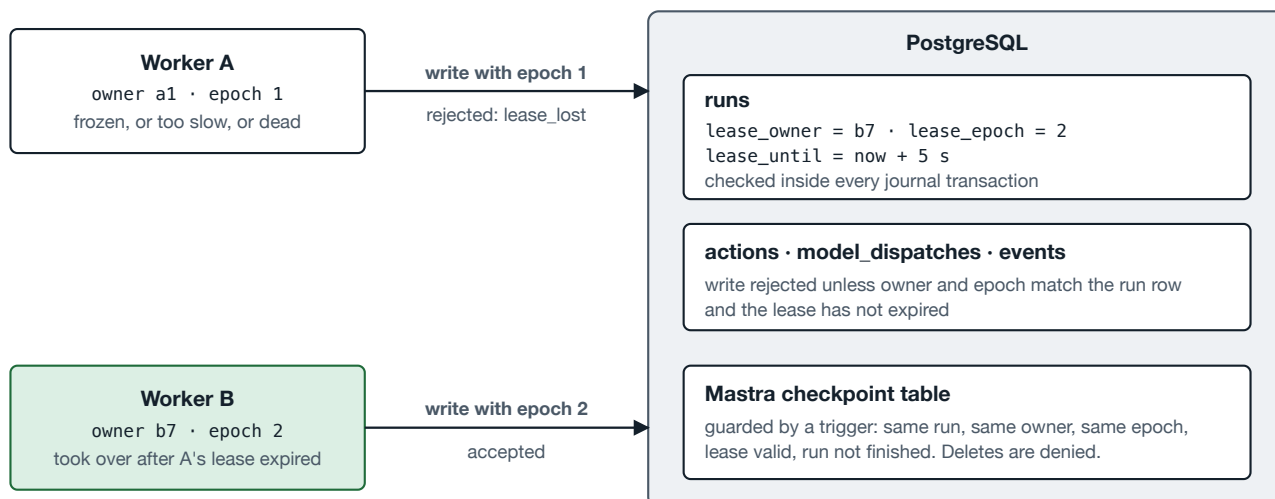
Imagine the tool was a request to a customer's server, or a write to a repository. If the process died after sending it, a retry could do the damage twice. A guess could report success that never happened. In a security product both are incidents. So the record says "uncertain", the run stops, and a person reconciles it. The counter fixture proves the mechanism with real kills before any real tool exists.

What "recorded" means

Each record is a PostgreSQL transaction that also appends an event and charges the budget. A model answer or tool result is stored with a SHA-256 digest and checked again on read. If the bytes changed, the read fails rather than returning altered evidence.

Crashes, leases and the fence

Only one worker may own a run at a time. Ownership is a lease: an owner ID and an epoch number stored on the run row. Every write checks both. When a worker dies, a new one takes a fresh lease with a higher epoch and continues from the journal. If the old worker is not really dead, it is fenced out.



Two fences: the journal's own checks, and a database trigger on Mastra's snapshot table so even the library's writes are checked.

Figure 4. A stale worker cannot advance state. The trigger makes this true even for Mastra's internal checkpoints.

The patched dependency

Real kill tests found two bugs in Mastra 1.67.0: on restart it chose the wrong step input, and it pruned model output it still needed. Ghidorah ships a byte-exact patch for both bundles. The installer only applies it when the original hashes match, and the backend re-checks the hashes before every run. An unpatched Mastra cannot execute.

Calling a real model

The model gateway is how Ghidorah talks to any language model. It makes exactly one dispatch per request, records it before and after, and never retries or falls back on its own. The first live route is GLM 4.7 through OpenRouter, pinned to a single upstream provider.

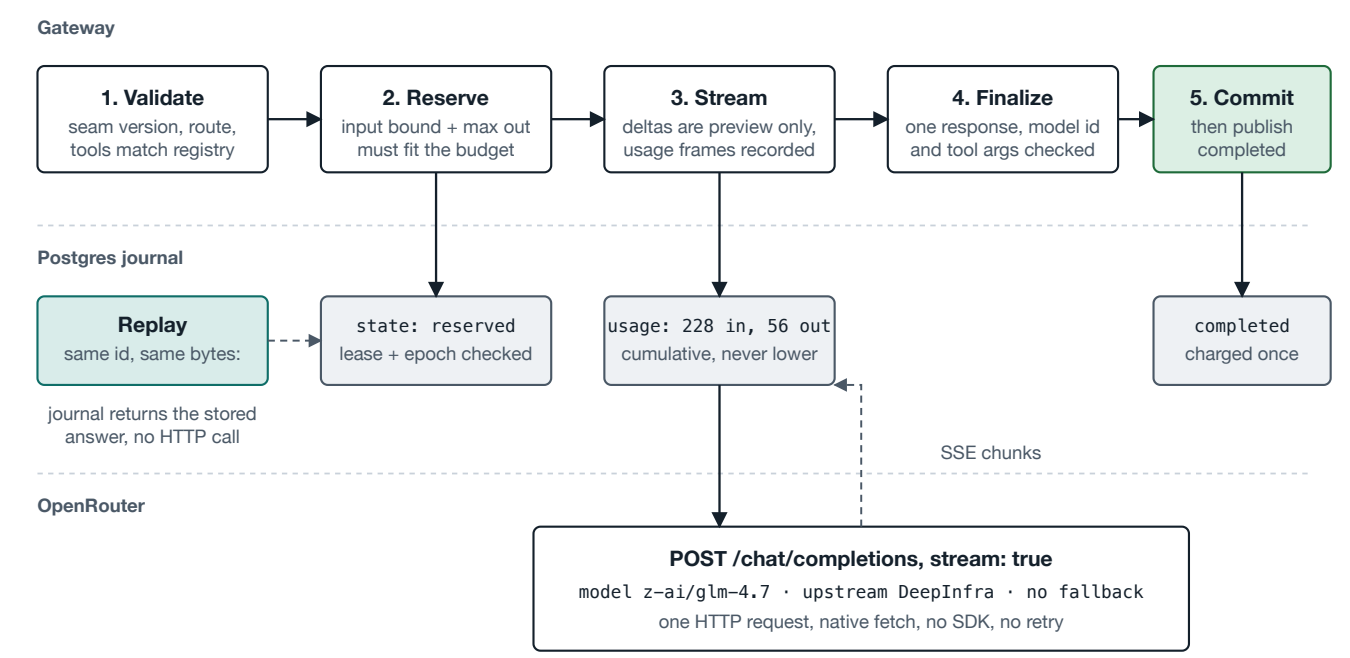


Figure 5. Reserve before dispatch, commit before publish. A replay of the same request never reaches the provider.

What the live test proved

CHECK	RESULT
Plain text answer	Passed
Tool call finalized in one HTTP request	Passed
Same request replayed from the journal, no provider call	Passed
1 ms timeout recorded as unresolved	Passed
Next dispatch blocked while one is unresolved	Passed
HTTP calls / estimated spend	2 / \$0.0005

Two lessons. OpenRouter spreads one model across several upstream providers, and one of them ignored the output cap; the gateway refused that response, so the route is now pinned. GLM 4.7 spends hidden reasoning tokens against the output cap, so budgets need headroom.

Where we are

Ghidorah is a tested control plane with no payload yet. The safety machinery is real; the product on top is not started. That order is deliberate for a security tool.

Built and tested

- Strict admission and versioned contracts
- Leases, heartbeats, wall-clock deadlines
- Authoritative journal with digest-checked evidence
- Recovery that blocks on uncertain actions
- Checkpoint fence trigger against stale workers
- Hash-pinned Mastra patch, verified before every run
- Model gateway, Postgres dispatch journal, OpenRouter adapter
- Live GLM 4.7 acceptance on a pinned route
- Shared contracts package for the frontend

Not built

- Real model wired into the Mastra loop (fixture still synthetic)
- Any tool beyond the counter: no repo read, no browser, no API
- A producer of findings; counts are zero by schema
- Independent verification (Gyms oracle) connected
- Sandboxed execution and network egress control
- Real target authorization and tenant isolation
- Network API, authentication, reconnect
- Approvals and human review at runtime

How this becomes code review and pentest

STEP	WHAT IT NEEDS	UNLOCKS
1	Wire the model gateway into the Mastra runtime	The counter fixture runs on GLM 4.7
2	Read-only repository tools behind the existing executor	Code review, closed world, no containment needed
3	A finding producer and persisted findings with receipts	The frontend's finding events light up
4	Sandbox, egress control, target authorization, Gyms verifier	Pentest

How to check any of this yourself

COMMAND	PROVES
<code>npm test</code>	131 unit tests, no database
<code>npm run test:integration</code>	26 tests with real process kills and fence races
<code>npm run eval</code>	100 catalogued cases with source fingerprints
<code>npm run repro:native</code>	Kill a native model call, recover from Postgres
<code>npm run acceptance:openrouter</code>	Live pinned-route model call, capped spend

A passing counter fixture is not production approval. The full list of release gates is in `docs/production-plan.md`.